

گزارش پروژه درس سیستم‌های نهفته

امیرمهدی طهماسبی بنیامین کریمی‌زاده کامران کریمی

۱۶ مرداد ۱۴۰۵

هدف اصلی این پروژه یک محیط شبیه سازی برای رایانش لبه خودرویی یا همان VEC. یعنی در واقع هر خودرو در طول حرکت خود در شهر، وظایف محاسباتی با حجم و زمان اجرای متفاوت تولید می کند. سامانه نهفته موجود در خودرو باید با در نظر گرفتن وضعیت فعلی خودرو، مدت وظیفه، میزان پوشش رادیویی هر سرور، ظرفیت پردازشی و صف منابع، یک زوج سرور و منبع را انتخاب کند. خروجی تصمیم به صورت یک شناسه یکتا از سرور و منبع پردازشی به خودرو بازگردانده می شود. یعنی باید انتخاب کند که وظیفه به کدام سرور و کدام منبع آن سرور باید برود. در کدی که ما زده‌ایم، آموزش، استنتاج و ارزیابی از هم جدا هستند. یعنی مدل در زمان تصمیم گیری به مسیر آینده یا داده خام دسترسی ندارد.

چهار مجموعه داده InTAS، Roma، TAPAS و T-Drive در آزمایش‌ها به کار رفته اند. روش‌های پایه شامل Greedy، Random و Uniform Random هستند و روش‌های پیشرفته از فیلتر Kalman، زنجیره Markov، راهبرد Coverage Load و شبکه GRU استفاده می کنند. معیارهای موفقیت، تاخیر، انرژی، طول صف، توازن بار و سربار پیش بینی برای دو وضعیت بار عادی و سنگین محاسبه شده اند. نتایج نشان می دهند که آگاهی از پوشش و صف، به ویژه در شبکه‌های پرتراکم، تاخیر و صف را به مقدار محسوسی کاهش می دهد.

۱ چستی مسئله

خودروهای هوشمند برای پردازش محیط، نقشه، هماهنگی ترافیک و سرویس‌ها به توان محاسباتی نیاز دارند. اجرای همه محاسبات درون خودرو هزینه سخت افزاری و انرژی را افزایش می دهد و ارسال همه داده‌ها به ابر نیز تاخیر ارتباطی زیادی دارد. رایانش لبه این فاصله را با قرار دادن سرورهای پردازشی نزدیک جاده کم می کند. اما خودرو در زمان پردازش ثابت نمی ماند. انتخاب سروری که در لحظه آغاز در دسترس است ممکن است چند ثانیه بعد و هنگام آماده شدن نتیجه، خارج از پوشش خودرو باشد.

در این پروژه، هر وظیفه دقیقاً به یک سرور لبه و یکی از منابع همان سرور سپرده می شود. هر منبع ظرفیت، توان مصرفی و صف مستقل دارد. بنابراین تصمیم مناسب فقط یافتن نزدیک ترین آنتن نیست؛ مدت اجرای وظیفه، مکان آینده خودرو، زمان انتظار و هزینه انرژی هم باید در نظر گرفته شوند.

حال هسته پروژه ما رفتار یک سامانه نهفته تصمیم گیر را شبیه‌سازی می کند. ورودی آن شناسه خودرو، زمان، مختصات، سرعت، زاویه حرکت و مدت وظیفه است. سامانه بعد از پیش بینی موقعیت، زوج `server_id:resource_id` را برمی گرداند. مدل در مرحله استنتاج به مسیر واقعی آینده دسترسی ندارد. مسیر آینده فقط نزد ارزیاب نگه داشته می شود تا پس از پایان پردازش مشخص شود خودرو هنوز در محدوده سرور انتخاب شده بوده است یا نه. چرخه کار کل پروژه ما به پنج بخش تقسیم می شود: خواندن و یکسان سازی داده، استقرار محل سرورها برای شبیه‌سازی، آموزش مدل‌ها، تصمیم گیری و ارزیابی. بنابراین نتیجه هر روش قابل تکرار و مقایسه می شود.

۲ معماری شبیه ساز

مدل وظیفه

وظیفه شماره i با بردار زیر نمایش داده می شود:

$$T_i = (a_i, c_i, d_i, b_i^+, b_i^-), \quad (۱)$$

که در آن a_i زمان ورود، c_i تعداد چرخه های محاسباتی، d_i مدت اسمی اجرا و دو مولفه آخر حجم داده ورودی و خروجی هستند. مدت های ۱۰ و ۳۰ ثانیه در ارزیابی اصلی استفاده شده اند. ورود وظایف از فرایند Poisson نمونه برداری می شود. در داده های مکانی کم نمونه مانند GPS، امکان ورود چند وظیفه بین دو مشاهده وجود دارد و زمان ورود آن ها در همان بازه پخش می شود.

مدل سرور و منابع چندگانه

هر سرور دارای مختصات (x_s, y_s) ، شعاع پوشش R_s ، نرخ ارسال و دریافت و مجموعه ای از منابع است. هر منبع ظرفیت f_r بر حسب میلیارد چرخه در ثانیه و توان فعال P_r دارد. زمان پردازش و انرژی فعال از روابط زیر به دست می آیند:

$$L_p = \frac{c_i}{f_r}, \quad (۲)$$

$$E_i = P_r L_p. \quad (۳)$$

تاخیر کل برابر است با:

$$L_i = L_u + L_g + L_q + L_p + L_d. \quad (۴)$$

که برابر است با مجموع تاخیر ارسال داده، در صف بودن داده، پردازش و بازارسال پاسخ. هر منبع فهرست کارهای زمان بندی شده خود را دارد. در نتیجه دو منبع روی یک سرور می توانند صف های متفاوتی داشته باشند. پروفایل های پیش فرض شامل eco CPU با ظرفیت ۸، balanced CPU با ظرفیت ۱۶ و accelerated GPU با ظرفیت ۳۲ میلیارد چرخه در ثانیه هستند. افزایش ظرفیت شتاب دهنده صف را کم می کند، اما توان فعال آن بیشتر است.

شرط موفقیت

فاصله خودرو و سرور در زمان t به شکل زیر محاسبه می شود:

$$D_{is}(t) = \sqrt{(x_i(t) - x_s)^2 + (y_i(t) - y_s)^2}. \quad (۵)$$

وظیفه زمانی موفق است که هم هنگام ارسال و هم در زمان تکمیل داخل شعاع پوشش باشد:

$$S_i = I[D_{is}(a_i) \leq R_s] I[D_{is}(a_i + L_i) \leq R_s]. \quad (۶)$$

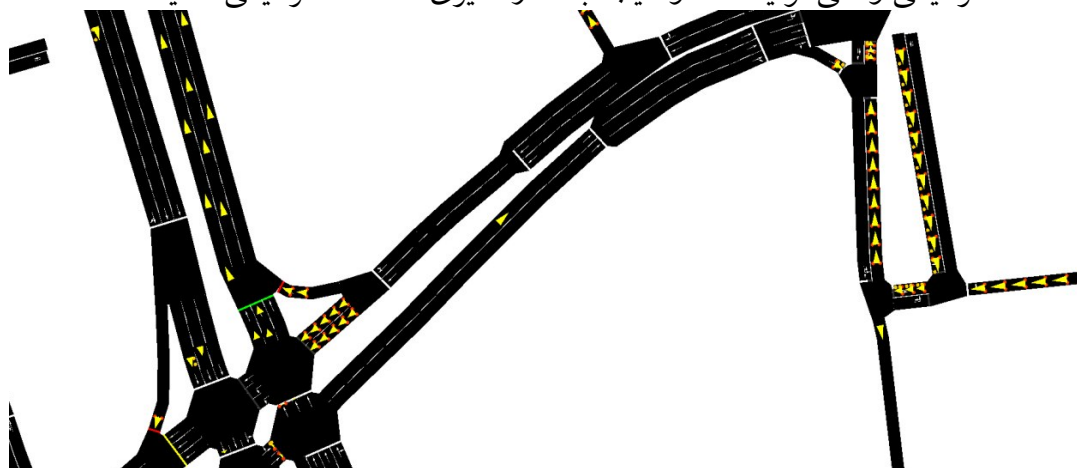
بعد از بررسی نتایج متوجه شدیم که موفقیت بسیار به صف وابسته است. حتی پیش بینی درست برای ۳۰ ثانیه، در صورت ایجاد صف چند دقیقه ای، دیگر کافی نیست. به همین دلیل ظرفیت های عادی تنظیم شده اند که صف وجود داشته باشد، ولی مزیت پیش بینی را کاملاً از بین نبرد.

۳ مجموعه داده های حرکتی

وجود چند شهر و چند نوع منبع داده، ارزیابی را از وابستگی به یک نقشه یا یک الگوی رفت و آمد دور می کند. داده های InTAS و TAPAS از سناریوهای ترافیکی SUMO می آیند، اما Roma و T-Drive مسیرهای واقعی تاکسی ها را ثبت کرده اند.

مجموعه داده InTAS

سناریوی InTAS به وسیله لوبو، نویمایر، فرناندز و فاکلی برای شهر اینگولشتات آلمان ساخته شده است [۱]. شبکه خیابان، ساختمان ها، ایستگاه های اتوبوس و چراغ های راهنمایی در آن مدل شده اند. تقاضای سفر از داده های جمعیتی و اطلاعات ترافیکی واقعی تولید شده و نتیجه با اندازه گیری ۲۴ نقطه ترافیکی مقایسه شده است.



شکل ۱: نمایی از خودروها و حرکت آن ها روی شبکه راه در شبیه ساز SUMO

مجموعه داده TAPAS

سناریوی TAPAS Cologne توسط مرکز هوافضای آلمان و جامعه توسعه SUMO منتشر شده است [۲]. تقاضای حرکت از عادت های سفر و زیرساخت منطقه کلن ساخته شده و مسیرها روی شبکه OpenStreetMap نگاشت شده اند. نسخه اصلی یک روز کامل را پوشش می دهد اما نسخه استفاده شده در این پروژه بازه پرتدد در صبح را در اختیار می گذارد. بزرگی شبکه و تعداد زیاد خودروها، این مجموعه را به آزمایش مناسبی برای صف های ناهمگون تبدیل می کند.



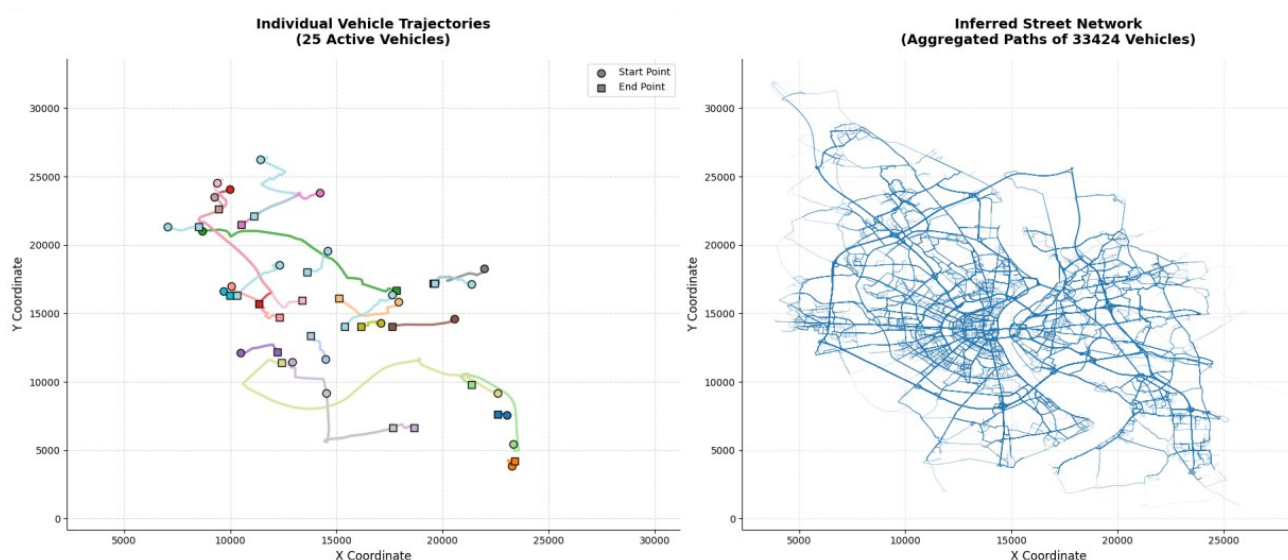
شکل ۲: نمایی از شهر در شبیه ساز SUMO

مجموعه داده روما

مجموعه Roma Taxi از مخزن CRAWDAD دریافت شده و توسط براچیاله، بونولا، لورتی، بیانکی، آمیچی و رابوفی فراهم شده است [۴]. داده شامل مختصات GPS حدود ۳۲۰ تاکسی در شهر رم طی ۳۰ روز است. مسیرهای واقعی این مجموعه، توقف های طولانی، نرخ نمونه برداری متغیر و الگوهای روزانه را وارد آزمایش می کنند؛ عواملی که در یک سناریوی کاملاً مصنوعی کمتر دیده می شوند.

مجموعه داده T-Drive

T-Drive توسط ژنگ و همکاران در Microsoft Research Asia منتشر شده است [۴]. این نمونه مسیر یک هفته ای ۱۰۳۵۷ تاکسی در پکن را با حدود ۱۵ میلیون نقطه و نزدیک به ۹ میلیون کیلومتر حرکت در بر می گیرد.



شکل ۳: سمت راست: شبکه خیابان استنباط شده از مسیرهای تجمیع شده؛ سمت چپ: نمونه مسیر خودروهای فعال

آماده سازی و نمونه گیری

در این کد ماژولی داریم که شناسه خودرو، زمان، مختصات، سرعت و زاویه را به قالب مشترک تبدیل می کند. داده های FCD به صورت جریانی خوانده می شوند و داده های خام GPS در Parquet ذخیره می شوند. مدل ها کل خط زمانی را یاد می گیرند.

برای ارزیابی Roma و T-Drive، به دلیل اینکه داده ها خیلی زیاد بودند، از ۱۲ پنجره زمانی در کل بازه استفاده شده است. تمام خودروهای فعال داخل هر پنجره حفظ می شوند تا شدت ترافیک و هم زمانی ورود وظایف از بین نرود. پنجره ها و تعداد نمونه های قبل و بعد از محدودسازی در فایل evaluation_sampling.json ثبت می شوند.

۴ روش استقرار سرورها

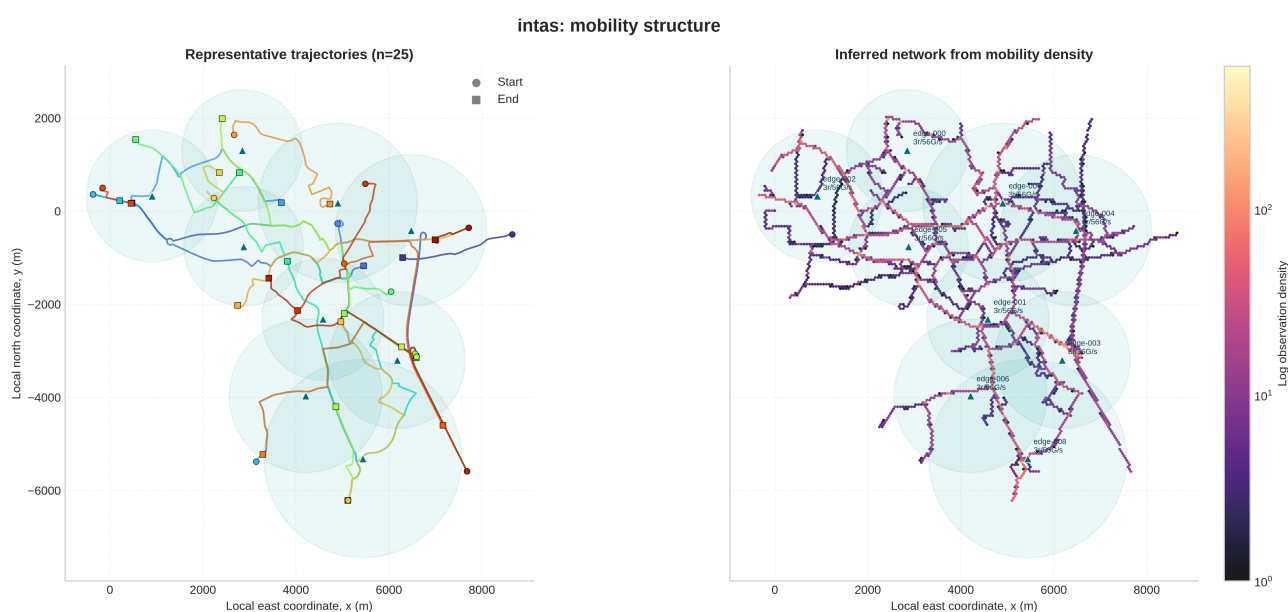
استقرار Grid

در روش Grid، مستطیل مرزی نقشه به بخش های منظم تقسیم می شود و سرورها در مرکز سلول ها قرار می گیرند. شعاع پوشش با قطر سلول و ضریب هم پوشانی تعیین می شود. این روش ساده است اما تراکم واقعی خودروها را نمی بیند. ممکن است بخشی از ظرفیت در ناحیه کم رفت و آمد قرار گیرد و یک بخش پرتردد با صف بلند روبه رو شود.

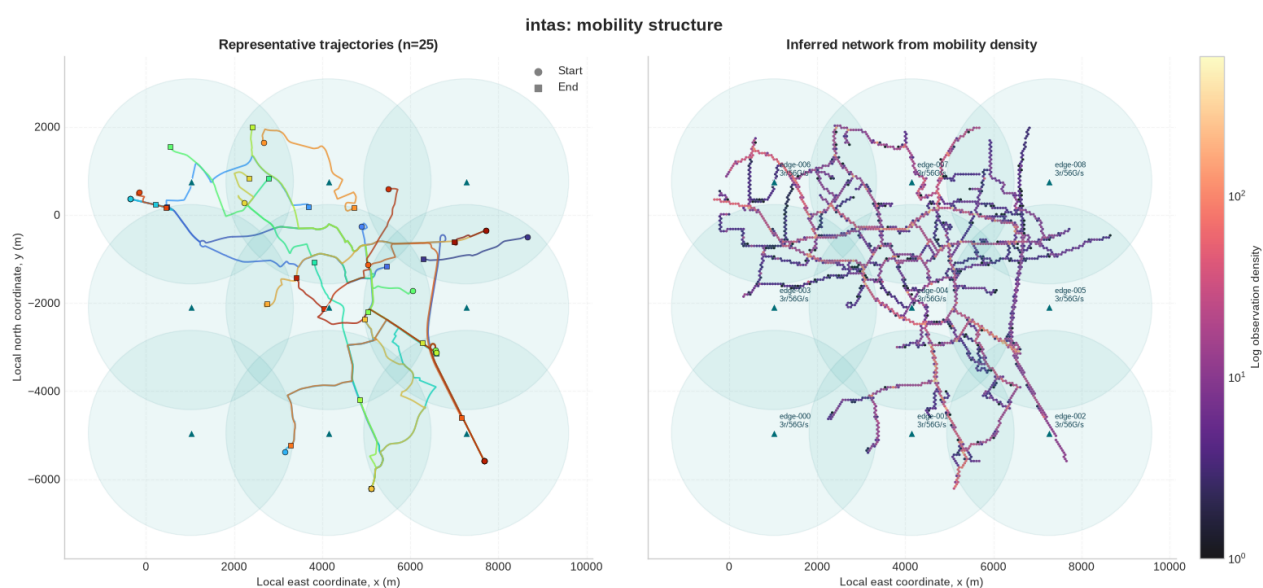
استقرار Density

در روش Density، مختصات مشاهده شده خوشه بندی می شوند و مرکز خوشه محل سرور است. شعاع هر سرور از صدک فاصله نقاط خوشه و ضریب هم پوشانی محاسبه می شود. در نتیجه منابع به نواحی پرتقاضا نزدیک تر هستند. این روش پوشش داده محور بهتری می دهد، ولی نتیجه آن به توزیع مجموعه داده وابسته است و باید همراه با فایل کامل محیط ثبت شود.

در داخل کد دستور place-servers این مرحله را از آموزش جدا می کند و یک فایل JSON شامل مرز نقشه، مکان سرورها، شعاع پوشش، پهنای باند و منابع پردازشی می سازد. کاربر می تواند فایل را پیش از آموزش ویرایش کند و سرورها را به شکل کاملاً دلخواهی قرار بدهد و تحلیل کند. اثر انگشت محیط نیز داخل مدل ذخیره می شود و ارزیابی مدل با محیط ناسازگار را رد می کند.



شکل ۴: توزیع سرور ها به روش Density



شکل ۵: توزیع سرور ها به روش Grid

۵ راهبردهای پیش بینی و انتخاب

روش Greedy

روش Greedy نزدیک ترین سروری را که در وضعیت فعلی خودرو پوشش دارد انتخاب می کند و سپس با توجه به صف، ظرفیت و انرژی، بهترین منبع همان سرور را برمی گزیند. هزینه محاسباتی آن کم است و برای وظایف کوتاه مناسب عمل می کند، اما افق آینده را نمی بیند. در سرعت بالا یا نزدیک مرز سلول، نزدیک ترین سرور فعلی ممکن است در زمان تکمیل قابل دسترس نباشد.

روش Random

روش Random ابتدا از میان سرورهایی که اکنون خودرو را پوشش می دهند یک سرور تصادفی و سپس یک منبع تصادفی انتخاب می کند. این روش آگاهی از آینده و صف ندارد، ولی شرط پوشش اولیه را رعایت می کند. بنابراین یک مبنای تصادفی منصفانه برای سنجش سود حاصل از مدل های دیگر است.

روش Uniform Random

روش Uniform Random از میان تمام زوج های سرور و منبع، بدون توجه به موقعیت، پوشش، صف، ظرفیت یا انرژی انتخاب می کند. این تعریف عمداً یک مبنای کاملاً ناآگاه است. توزیع بار آن معمولاً یکنواخت می شود، اما بیشتر انتخاب ها ممکن است از ابتدا خارج از پوشش باشند. نرخ موفقیت پایین این روش خطا نیست؛ نشان می دهد توازن تعداد کارها به تنهایی معیار کافی برای برون سپاری نیست.

فیلتر Kalman

مدل Kalman وضعیت حرکت را با بردار مکان و سرعت نمایش می دهد:

$$\vec{x}_k = [x_k \ y_k \ v_{x,k} \ v_{y,k}]^T. \quad (7)$$

مرحله پیش بینی و اصلاح چنین است:

$$\hat{\vec{x}}_{k|k-1} = F \hat{\vec{x}}_{k-1|k-1}, \quad (8)$$

$$K_k = P_{k|k-1} H^T (H P_{k|k-1} H^T + R)^{-1}, \quad (9)$$

$$\hat{\vec{x}}_{k|k} = \hat{\vec{x}}_{k|k-1} + K_k (\vec{z}_k - H \hat{\vec{x}}_{k|k-1}). \quad (10)$$

پارامترهای نگهداشت سرعت و واریانس نویز از کل داده آموزش استخراج می شوند. موقعیت به اندازه مدت وظیفه به جلو برده می شود و زوجی انتخاب می شود که پوشش فعلی و آینده را با هزینه صف و انرژی ترکیب کند. مزیت این روش سرعت زیاد است؛ ضعف آن فرض حرکت تقریباً خطی در تقاطع ها و تغییر مسیرهای ناگهانی است.

مدل Markov

در مدل Markov، نقشه به سلول های گسسته تقسیم می شود و حرکت خودرو به دنباله حالت ها تبدیل می شود. احتمال انتقال از حالت i به j از شمارش کل گذارها به دست می آید:

$$P_{ij} = \frac{N_{ij}}{\sum_j N_{ij}}. \quad (11)$$

برای پیش بینی چند گام، انتقال های محتمل به اندازه افق وظیفه دنبال می شوند. مدل تغییر مسیرهای تکراری را بهتر از حرکت خطی می آموزد و حجم ذخیره آن کم است. کیفیت آن به اندازه شبکه وابسته است: شبکه خیلی درشت جزئیات خیابان را حذف می کند و شبکه خیلی ریز گذارهای کم مشاهده و پراکنده می سازد.

راهبرد Coverage Load

راهبرد Coverage Load یک روش ابتکاری آگاه از پوشش و صف است. مکان آینده از بردار سرعت فعلی برآورد می شود. سپس برای هر زوج سرور و منبع، جریمه پوشش فعلی، فاصله آینده، زمان انتظار، زمان خدمت، ظرفیت و انرژی محاسبه می شود. شکل کلی امتیاز به صورت زیر است:

$$J(s, r) = w_c C(s) + w_q Q(r) + w_p L_p(r) + w_e E(r) + w_d D(s). \quad (12)$$

زوج با کمترین J انتخاب می شود. این روش آموزش سنگین ندارد، اما با دیدن وضعیت لحظه ای صف می تواند بار را از منابع شلوغ دور کند. نتایج نشان می دهند که در بار عادی و سنگین، این ویژگی تاخیر و صدک نود و پنجم صف را به طور محسوسی کاهش می دهد.

شبکه GRU

شبکه GRU برای پیش بینی دنباله زمانی موقعیت به کار می رود. ورودی هر گام شامل حرکت نسبی، سرعت، زاویه و فاصله زمانی است. دو دروازه اصلی رفتار حافظه را کنترل می کنند. دروازه به روزرسانی تعیین می کند چه مقدار از حافظه پیشین باقی بماند و دروازه بازنشانی میزان اثر گذشته بر حالت پیشنهادی را تنظیم می کند:

$$z_t = \sigma(W_z x_t + U_z h_{t-1} + b_z), \quad (13)$$

$$r_t = \sigma(W_r x_t + U_r h_{t-1} + b_r), \quad (14)$$

$$\tilde{h}_t = \tanh(W_h x_t + U_h(r_t \odot h_{t-1}) + b_h), \quad (15)$$

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t. \quad (16)$$

مدل با پنجره های دنباله ای از سراسر داده آموزش می بیند و وزن ها در پوشه saved_models ذخیره می شوند. هنگام استنتاج، خروجی یک گام دوباره به مدل داده می شود تا جابه جایی تا انتهای مدت وظیفه ساخته شود. موقعیت نهایی وارد همان انتخاب مشترک پوشش، منبع، صف و انرژی می شود. GRU توان یادگیری پیچ و تغییر سرعت را دارد، اما به داده، زمان آموزش و کتابخانه PyTorch نیازمند است و سربار آن از روش های دیگر بیشتر است.

۶ معیارهای ارزیابی

معیارهای زیر برای هر الگوریتم و هر وظیفه ثبت شده اند:

- نرخ موفقیت پوشش در آغاز و زمان تکمیل؛
- میانگین و صدک نود و پنجم تاخیر انتها به انتها؛
- میانگین، صدک نود و پنجم و بیشینه زمان صف؛
- طول صف در لحظه تخصیص؛
- انرژی فعال هر وظیفه و انرژی کل؛

– توزیع بار میان سرورها و زوج های سرور و منبع؛

– میانگین و صدک نود و پنجم سربار زمانی پیش بینی.

همه الگوریتم ها دقیقا همان وظایف و حقیقت آینده را دریافت می کنند و محیط زمان اجرا برای هر الگوریتم از حالت خالی ساخته می شود. وظایفی که زمان تکمیل آن ها بیرون از مسیر ثبت شده باشد در مخرج نرخ موفقیت قرار نمی گیرند؛ بنابراین تعداد وظایف قابل ارزیابی باید کنار درصد موفقیت خوانده شود.

۷ نتایج عددی

در این بخش، نتایج کامل اجرای الگوریتم های مختلف بر روی هر چهار مجموعه داده به صورت مجزا آورده شده است.

عملکرد الگوریتم ها در مجموعه داده InTAS

جدول؟؟ نشان می دهد که در این محیط، الگوریتم های آگاه از وضعیت توانسته اند زمان انتظار را کاهش دهند.

جدول ۱: جزئیات عملکرد تمام الگوریتم ها در مجموعه داده InTAS

الگوریتم	نرخ موفقیت	میانگین تاخیر (s)	صدک ۹۵ صف (s)	انرژی (J)	سربار پیش بینی (ms)
Coverage Load	96.09%	15.307	7.045	1468.38	0.102
Kalman	96.09%	15.489	7.210	1460.38	0.064
Greedy	95.89%	15.729	8.490	1472.76	0.012
Markov	95.51%	15.419	7.045	1459.05	3.857
Random	95.26%	19.551	29.347	1567.24	0.003
Furthest	63.46%	316.068	1256.637	1353.24	0.002
Uniform Random	13.75%	19.667	30.217	1567.43	0.003

عملکرد الگوریتم ها در مجموعه داده Roma

همان طور که در جدول؟؟ مشاهده می شود، روش حریصانه به دلیل انتخاب نزدیک ترین سرور دچار گلوگاه شده است، در حالی که راهبرد Coverage Load این مشکل را برطرف کرده است.

جدول ۲: جزئیات عملکرد تمام الگوریتم ها در مجموعه داده Roma

الگوریتم	نرخ موفقیت	میانگین تاخیر (s)	صدک ۹۵ صف (s)	انرژی (J)	سربار پیش بینی (ms)
Greedy	98.38%	32.938	102.704	1548.82	0.013
Coverage Load	98.35%	15.290	8.819	1481.44	0.109
Kalman	98.33%	15.947	9.887	1458.13	0.055
Markov	98.21%	15.947	14.929	1533.91	0.093
Random	97.83%	44.017	156.008	1518.27	0.004
Uniform Random	40.81%	26.348	61.383	1529.13	0.003

عملکرد الگوریتم ها در مجموعه داده TAPAS

در جدول؟؟ مشخص است که روش های فاقد پیش بینی به دلیل تولید صف های عظیم، تاخیرهای نجومی ایجاد کرده اند، در حالی که فیلتر Kalman پایداری سیستم را حفظ کرده است.

جدول ۳: جزئیات عملکرد تمام الگوریتم‌ها در مجموعه داده TAPAS

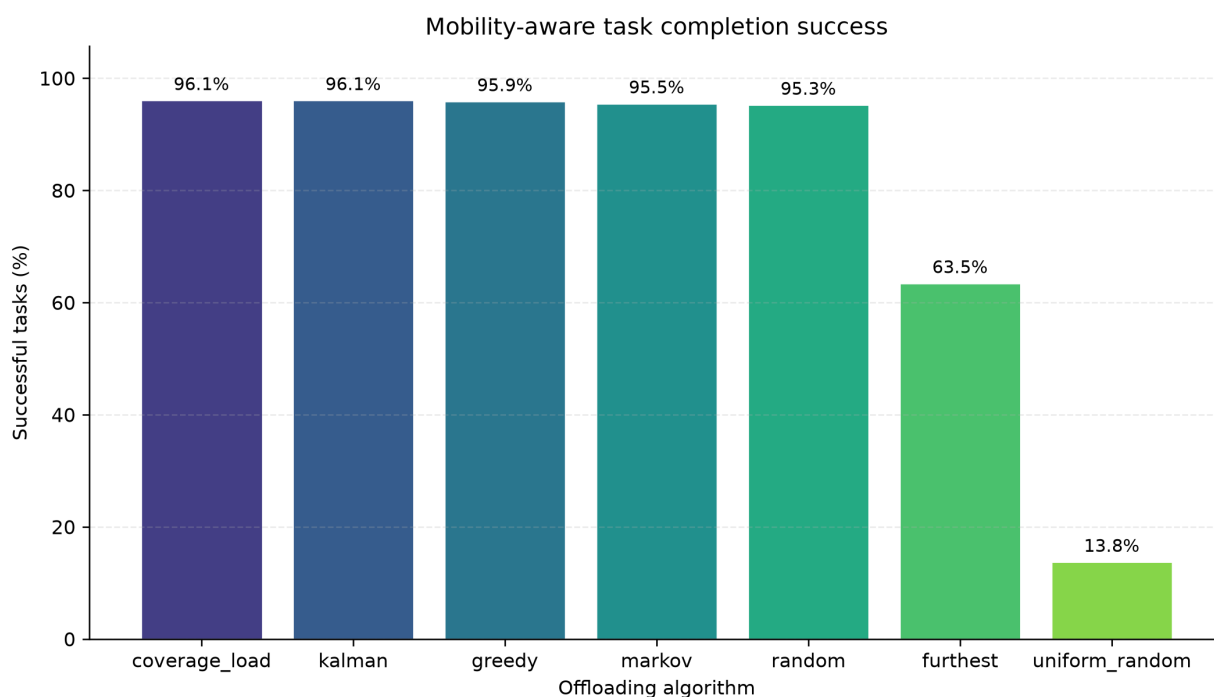
الگوریتم	نرخ موفقیت	میانگین تاخیر (s)	صدک ۹۵ صف (s)	انرژی (J)	سر بار پیش‌بینی (ms)
Coverage Load	97.73%	20.194	40.334	1560.50	0.107
Kalman	97.60%	19.469	36.663	1557.33	0.068
Markov	97.16%	20.168	39.561	1557.87	5.245
Greedy	96.82%	328.239	2319.308	1564.49	0.012
Random	91.27%	909.408	5074.045	1502.74	0.004
Furthest	65.70%	14130.727	31191.110	1302.66	0.003
Uniform Random	18.66%	675.902	3571.145	1507.45	0.003

عملکرد الگوریتم‌ها در مجموعه داده T-Drive

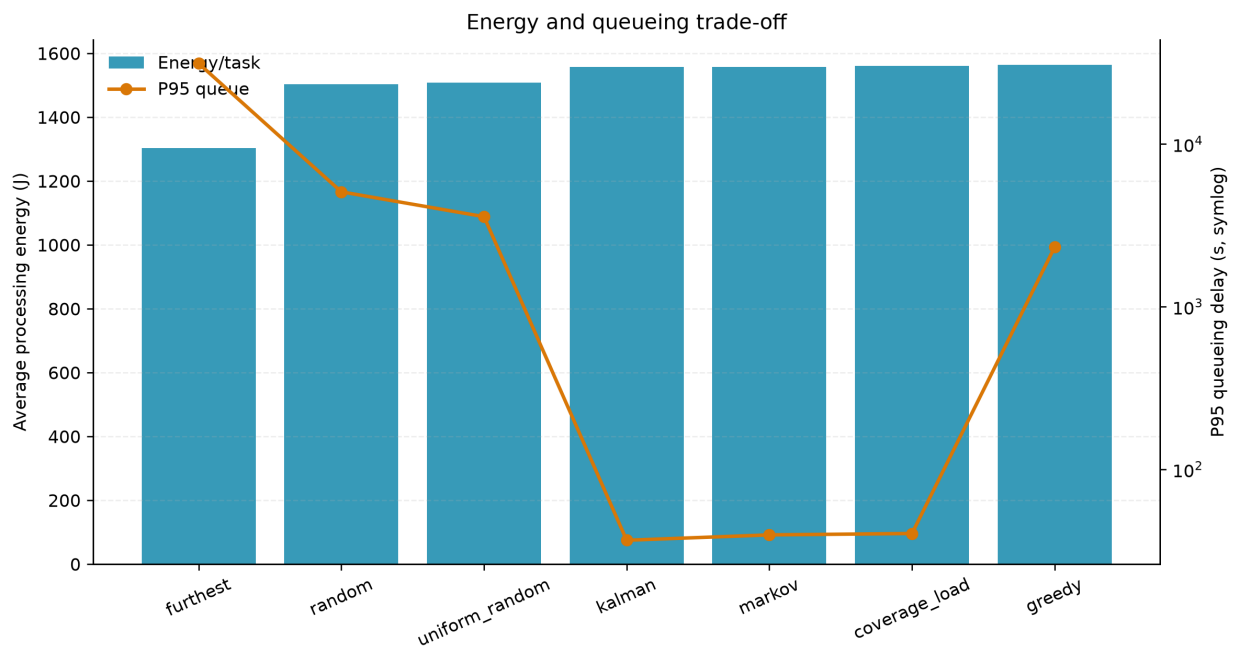
جدول ۴؟ نشانگر یک بار پردازشی به شدت سنگین است. در اینجا، تفاوت میانگین تاخیر الگوریتم Coverage Load با سایر روش‌ها نمایانگر توانایی آن در توزیع هوشمندانه بار در شرایط بحرانی است.

جدول ۴: جزئیات عملکرد تمام الگوریتم‌ها در مجموعه داده T-Drive

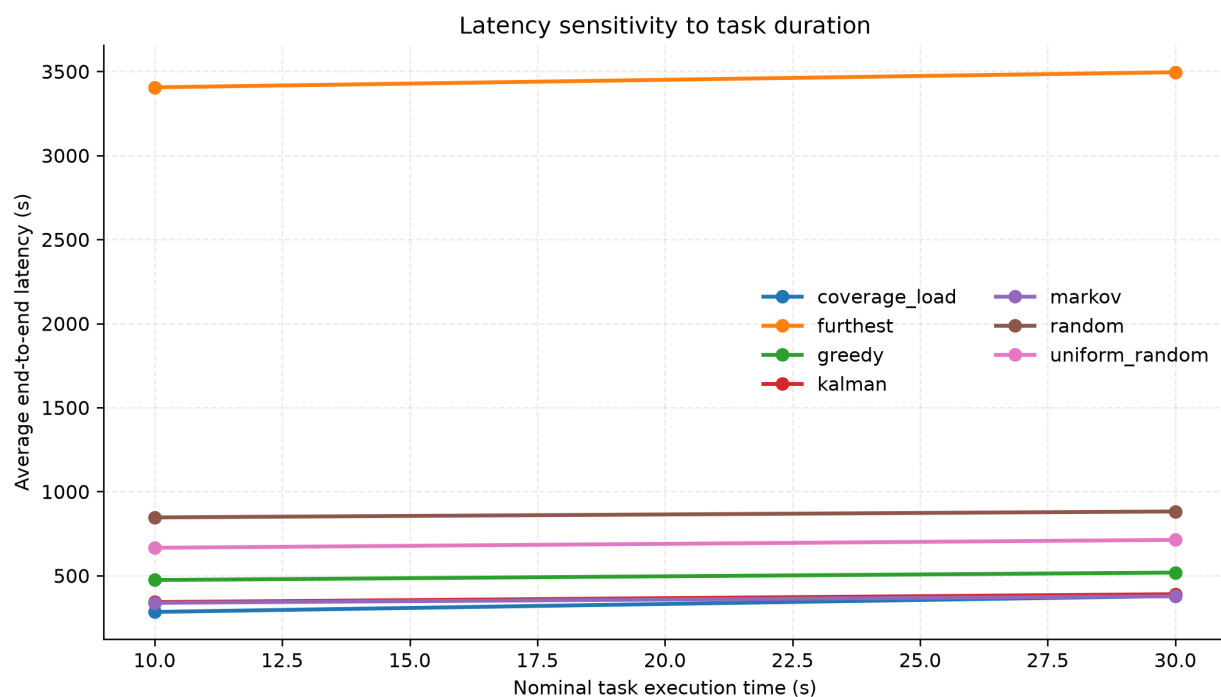
الگوریتم	نرخ موفقیت	میانگین تاخیر (s)	صدک ۹۵ صف (s)	انرژی (J)	سر بار پیش‌بینی (ms)
Markov	100.00%	4430.767	10375.935	1616.95	0.110
Greedy	99.78%	1742.419	5713.223	1615.01	0.013
Kalman	99.50%	875.894	2027.177	1615.99	0.076
Random	99.47%	1500.305	5012.748	1503.77	0.003
Coverage Load	99.19%	821.992	1846.654	1593.64	0.151
Uniform Random	73.30%	1246.798	3860.591	1502.91	0.003



شکل ۶: نرخ موفقیت الگوریتم‌ها برای InTAS در بار عادی



شکل ۷: بررسی انرژی و صف در مجموعه TAPAS



شکل ۸: اثر مدت وظیفه بر تاخیر در بار عادی

تحلیل داده‌های به دست آمده از چهار محیط شبیه‌سازی، الگوهای رفتاری مهمی را نشان می‌دهد.

ناکارآمدی روش‌های حریصانه در ترافیک متراکم

نتایج نشان می‌دهند که الگوریتم Greedy، با وجود این که در داده‌های Roma بالاترین نرخ موفقیت را ثبت کرده است، اما مستعد ایجاد گلوگاه پرداشی است. چون خودروها صرفاً نزدیک‌ترین سرور را انتخاب می‌کنند که باعث می‌شود سرورهای واقع در تقاطع‌های شلوغ یا مرکز شهر، صف‌های طولانی (بیش از 102 ثانیه در Roma و 2319 ثانیه در TAPAS) را تجربه کنند. پس در نظر نگرفتن وضعیت صف، عملاً مزیت مجاورت فیزیکی آنتن را از بین می‌برد و تاخیر کل را افزایش می‌دهد.

توازن بی‌فایده بار در توزیع یکنواخت

روش Uniform Random در تمامی محیط‌ها کمترین ضریب تغییرات (CV) را ثبت کرده است که نشان‌دهنده توزیع کاملاً مساوی و ایده‌آل وظایف بین تمام منابع است. اما این توازن بار به قیمت از دست رفتن فاجعه‌بار نرخ موفقیت (تا حد 13.75% در InTAS) تمام شده است.

عملکرد Kalman

در میان روش‌های مبتنی بر پیش‌بینی مسیر، فیلتر Kalman عملکردی بهینه از خود نشان داده است. این الگوریتم در داده‌های بسیار متراکم TAPAS با ثبت کمترین میانگین تاخیر (19.46 ثانیه) و صف (36.66 ثانیه)، توانسته است بحران ترافیک را مدیریت کند، در حالی که سربار محاسباتی آن تنها 0.068 میلی‌ثانیه برای هر تصمیم بوده است. اما مدل Markov با وجود موفقیت قابل قبول، در برخی محیط‌ها مانند TAPAS سربار پرداشی بیش از 5 میلی‌ثانیه داشته است.

References

- [1] S. C. Lobo, S. Neumeier, E. M. G. Fernandez, and C. Facchi, "InTAS – The Ingolstadt Traffic Scenario for SUMO," SUMO User Conference, 2020. <https://arxiv.org/abs/2011.11995>
- [2] Eclipse SUMO Documentation, "TAPASCologne Scenario." <https://sumo.dlr.de/docs/Data/Scenarios/TAPASCologne.html>
- [3] L. Bracciale, M. Bonola, P. Loreti, G. Bianchi, R. Amici, and A. Rabuffi, "CRAWDAD Roma Taxi Mobility Traces," 2014. <https://crawdad.org/roma/taxi/20140717/>
- [4] Y. Zheng, "T-Drive Trajectory Data Sample," Microsoft Research, 2011. <https://www.microsoft.com/en-us/research/publication/t-drive-trajectory-data-sample/>